
Assignment 01: Linux Kernel Compilation & Parallelism Benchmarking

Course: Parallel and Distributed Computing (Spring 2026)

Due Date: February 1, 2026 (11:59 PM)

Points: 25

Estimated Time to Complete: 3 – 5 Hours (includes waiting time for compilation)

1. Objectives

By completing this assignment, you will:

1. **Establish a Development Environment:** Set up a working Linux environment for future programming assignments.
 2. **Customize the Kernel:** Modify kernel configuration and source files to understand how the OS is structured.
 3. **Witness Parallelism:** Quantify the benefits of parallel computing by contrasting single-threaded vs. multi-threaded compilation performance.
-

2. Prerequisites

- **Hardware:** A laptop/PC with internet access.
 - **Software:** VirtualBox or VMware (Recommended) OR a native Linux installation.
 - **Resource Allocation:** If using a Virtual Machine, allocate at least **4 CPU cores** and **40GB of storage**.
-

3. The Tasks

Step 1: Set Up Your Linux Box (5 Points)

Install a Linux distribution. Recommended: **Ubuntu 24.04 LTS** or **Fedora Workstation 43**.

Step 2: Prepare the Build Environment

You need compilers and libraries. Run the following commands for your specific distribution:

For Ubuntu / Debian / Mint:

```
# Bash
```

```
sudo apt update
sudo apt install build-essential libncurses-dev bison flex libssl-dev libelf-dev
```

For Fedora / Red Hat / CentOS:

```
# Bash
```

```
sudo dnf groupinstall "Development Tools"
sudo dnf install ncurses-devel bison flex elfutils-libelf-devel openssl-devel
```

Step 3: Get the Kernel Source

1. Go to kernel.org and download the **latest stable kernel (for example: 6.18.6)** tarball.
2. Extract the source code in your home directory using following commands:
Bash
tar -xvf linux-6.x.x.tar.xz
cd linux-6.x.x

Step 4: Customization (Important!)

Task A: The Configuration Change (Useful Tuning)

We will optimize the kernel for desktop responsiveness.

1. Copy your existing config: `cp /boot/config-$(uname -r) .config`
2. Open the configuration menu:
Bash
make menuconfig
3. Navigate to: **Processor type and features > Timer frequency.**
4. Ensure **1000 HZ** is selected. (This makes the system check for interrupts more often, making the UI feel snappier on a desktop system).
5. Save and Exit.

Task B: The File Modification (Your Signature)

We will modify the build file so your name is permanently embedded in the kernel version.

1. Open the file named Makefile (in the root of the kernel folder) using a text editor (like nano or gedit).
2. Look for the line at the very top that says `EXTRAVERSION =`.
3. Change it to include your name (no spaces). For example:
Makefile
`EXTRAVERSION = -tom-riddle-linux`

4. Save the file. When you eventually boot this kernel, `uname -r` will show something like

6.8.0-tom-riddle-linux.

Step 5: The Experiment (15 Points)

Experiment A: Single-Core Compilation (The Baseline)

1. Run the build on a single core.

```
# Bash
time make
```

2. **Requirement:** You **must** let this process finish completely to get accurate data. On a single core, this may take 1.5 to 3 hours depending on your machine speed. Go grab a coffee or study for another class while this runs.
3. Record the real time output once it finishes.

Experiment B: Multi-Core Compilation (The Parallelism)

1. **Crucial Step:** You must clean the build artifacts from the previous run, or the second test will finish instantly and your data will be wrong.

```
# Bash
make clean
```

2. Run the build using all available cores.

```
# Bash
time make -j$(nproc)
```

3. Wait for the compilation to finish. Record the real time output.

Step 6: Installation & Boot (5 Points)

Once Experiment B is done, install the modules and the kernel image:

```
# Bash
```

```
sudo make modules_install
sudo make install
```

Reboot your machine. Select your new kernel from the boot menu. Verify your success by opening a terminal and typing:

```
Bash
```

```
uname -r
```

You should see your name in the output!

4. Deliverables

Submit a single PDF report containing:

1. **System Specs:** A screenshot of the `lscpu` command.
2. **The Proof:** A screenshot of your terminal showing `uname -r` displaying the kernel version **with your name appended**.
3. **The Data:**
 - Time taken for Single-Core (Full execution).
 - Time taken for Multi-Core.
4. **The Analysis:**
 - Calculate the **Speedup** ($T_{\text{serial}} / T_{\text{parallel}}$).
 - Compare the speedup to your core count. Is it linear? (e.g., did 4 cores make it exactly 4x faster?) Explain why or why not.

Tips for Success

- **Storage Space:** Compilation generates massive files. Ensure you have **20GB+ free space** inside your Linux environment before starting.
- **Patience is Key:** The single-core compilation (Step 5A) will be slow. This is intentional; it allows you to "feel" the cost of serial processing.
- **The "localmodconfig" Trick:** If `make menuconfig` is overwhelming, you can run `make localmodconfig` *before* Step 4 Task A. This disables drivers for hardware you don't have, significantly speeding up the compilation time.